

LEVEL 13 — AGGREGATION (Pandas)

Beginner-friendly, line-by-line examples with outputs

1. What is Aggregation?

Aggregation means combining many values into one summary value, such as sum, mean, minimum, maximum, or count.

```
import pandas as pd

df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR"],
    "Salary": [50000, 60000, 40000, 45000]
})

print(df["Salary"].sum())
print(df["Salary"].mean())
```

Output:

```
150000.0
48750.0
```

2. agg()

`agg()` applies one or more aggregation functions to a Series or DataFrame.

```
import pandas as pd

df = pd.DataFrame({
    "Salary": [50000, 60000, 40000, 45000]
})

result = df["Salary"].agg("mean")
print(result)
```

Output:

```
48750.0
```

3. Multiple Functions

You can pass a list of functions to `agg()`.

```
result = df["Salary"].agg(["sum", "mean", "min", "max"])
print(result)
```

Output:

```
sum      195000
mean      48750
min       40000
max       60000
Name: Salary, dtype: int64
```

4. Custom Aggregation

A custom aggregation is your own function. Here, we calculate the salary range.

```
def salary_range(x):
    return x.max() - x.min()
```

```
result = df["Salary"].agg(salary_range)
print(result)
```

Output:

```
20000
```

5. Column-wise Aggregation

With axis=0, Pandas works down each column. This is the default for DataFrame aggregation.

```
df = pd.DataFrame({
    "Salary": [50000, 60000, 40000],
    "Bonus": [5000, 6000, 4000]
})

result = df.agg("sum", axis=0)
print(result)
```

Output:

```
Salary    150000
Bonus      15000
dtype: int64
```

6. axis=0

axis=0 means aggregate vertically, column by column.

```
result = df.agg("mean", axis=0)
print(result)
```

Output:

```
Salary    50000.0
Bonus      5000.0
dtype: float64
```

7. Row-wise Aggregation

With axis=1, Pandas works across columns for each row.

```
result = df.agg("sum", axis=1)
print(result)
```

Output:

```
0    55000
1    66000
2    44000
dtype: int64
```

8. axis=1

axis=1 means aggregate horizontally, row by row.

```
result = df.agg("mean", axis=1)
print(result)
```

Output:

```
0    27500.0
1    33000.0
2    22000.0
dtype: float64
```

9. Named Aggregation

Named aggregation lets you choose the output column name and aggregation function.

```
df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR"],
    "Salary": [50000, 60000, 40000, 45000]
})

result = df.groupby("Department").agg(
    Average_Salary=("Salary", "mean")
)

print(result)
```

Output:

Average_Salary	Department
42500.0	HR
55000.0	IT

10. Multiple Named Aggregations

You can calculate several summaries and give each result a meaningful name.

```
result = df.groupby("Department").agg(
    Total_Salary=("Salary", "sum"),
    Average_Salary=("Salary", "mean"),
    Minimum_Salary=("Salary", "min"),
    Maximum_Salary=("Salary", "max"),
    Employee_Count=("Salary", "count")
)

print(result)
```

Output:

Total_Salary	Average_Salary	Minimum_Salary	Maximum_Salary	Employee_Count
85000	42500.0	40000	45000	2
110000	55000.0	50000	60000	2

11. Grouped Aggregation

GroupBy combined with agg() is one of the most useful Pandas patterns for data analysis.

```
result = df.groupby("Department")["Salary"].agg(["sum", "mean", "min", "max", "count"])
print(result)
```

Output:

sum	mean	min	max	count
85000	42500.0	40000	45000	2
110000	55000.0	50000	60000	2

12. Multiple Columns with agg()

You can apply different aggregation functions to different columns.

```
df = pd.DataFrame({
    "Sales": [100, 200, 300],
    "Profit": [20, 50, 80]
})
```

```

result = df.agg({
    "Sales": ["sum", "mean"],
    "Profit": ["min", "max"]
})

print(result)

```

Output:

```

Sales  Profit
sum    600.0    NaN
mean   200.0    NaN
min     NaN    20.0
max     NaN    80.0

```

13. Custom Aggregation on a DataFrame

You can create a custom function and apply it to columns.

```

def data_range(x):
    return x.max() - x.min()

result = df.agg(data_range)
print(result)

```

Output:

```

Sales      200
Profit      60
dtype: int64

```

14. Practical Example

This example combines grouping, multiple functions, and named aggregation.

```

df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR", "Sales"],
    "Salary": [50000, 60000, 40000, 45000, 55000],
    "Bonus": [5000, 6000, 4000, 4500, 5500]
})

result = df.groupby("Department").agg(
    Employees=("Salary", "count"),
    Avg_Salary=("Salary", "mean"),
    Total_Bonus=("Bonus", "sum"),
    Max_Salary=("Salary", "max")
)

print(result)

```

Output:

```

Employees  Avg_Salary  Total_Bonus  Max_Salary
Department
HR          2         42500.0         8500         45000
IT          2         55000.0        11000         60000
Sales       1         55000.0         5500         55000

```

Quick Revision

Concept	Meaning
agg()	Apply one or more aggregation functions

Multiple functions	Use a list such as ['sum', 'mean', 'max']
Custom aggregation	Use your own Python function
axis=0	Work down columns / column-wise
axis=1	Work across rows / row-wise
Named aggregation	Choose meaningful output names
Grouped aggregation	groupby() + agg()